Project: Building a Simple Java HTTP Server

Introduction:

This document describes how to build a basic HTTP server in Java using the built-in com.sun.net.httpserver package. This server will be able to handle incoming HTTP requests, process them, and return responses. It's a simple example, but it demonstrates the fundamental principles of server-side web development in Java.

Key Components and Concepts:

- **HTTP (Hypertext Transfer Protocol):** The foundation of data communication on the web. It defines how clients (like web browsers) and servers interact. Key aspects include:
    - **Requests:** Clients send requests to servers, specifying a *method* (e.g., GET, POST), a *URI* (Uniform Resource Identifier, like a URL path), and *headers* (metadata about the request).
    - **Responses:** Servers send back responses, including a *status code* (e.g., 200 OK, 404 Not Found), *headers* (metadata about the response), and a *body* (the actual content, like HTML, JSON, or text).
- **com.sun.net.httpserver Package:** A package included with standard Java (though technically in the com.sun namespace, which means it's not *guaranteed* to be present on all Java implementations, but it is very widely available). This package provides a simple, lightweight way to create HTTP servers without needing external libraries.
- **HttpServer Class:** The core class for creating the server. You create an instance of this class, configure it, and then start it to listen for connections.
- **InetSocketAddress Class:** Represents a network address, combining an IP address and a port number. You use this to tell the HttpServer where to listen.
    - **Port Number** Select an available port, commonly 8000 for development.
    - **IP Address:** When creating the InetSocketAddress, if you just provide the port, the server will listen on all available network interfaces (the "wildcard" address).
- **HttpHandler Interface:** Defines how to *handle* an incoming HTTP request. You create a class that *implements* this interface. The key method is handle(HttpExchange exchange).
- **HttpExchange Class:** Represents a single request/response pair. It provides methods to:
    - Get information about the request (method, URI, headers, body).
    - Set the response status code, headers, and body.

- **Context:** A mapping between a URL path (like "/hello") and an HttpHandler. When a request comes in for a particular path, the associated HttpHandler is invoked. You create contexts using the createContext() method of the HttpServer.
- **Executor:** (Optional) An object that manages how the server handles multiple requests. For simple servers, you can use the default executor (by setting the executor to null). More complex servers might use thread pools for better performance.
- **Request Method:** The HTTP verb (GET, POST, PUT, DELETE, etc.) that indicates the desired action.
- **Request URI:** The path part of the URL that identifies the requested resource (e.g., "/products/123").
- **Response Status Code:** A three-digit code that indicates the outcome of the request (e.g., 200 OK, 404 Not Found, 500 Internal Server Error).
- **Response Headers:** Metadata about the response, such as the content type (e.g., "text/html", "application/json") and content length.
- **Response Body:** The actual content of the response (e.g., the HTML of a web page, the JSON data of an API response).
- **OutputStream:** A stream used for writing data. In this case, you'll use the OutputStream from the HttpExchange to send the response body back to the client. *Always remember to close the OutputStream when you're finished.*
- **Exception Handling (IOException):** Network operations can fail. Your code should be prepared to handle IOExceptions, which signal problems like network errors or port conflicts.

General Development Process:

1. **Setup:**
    - Create a new Java project.
    - Import the necessary classes from com.sun.net.httpserver, java.net, and java.io.
2. **Create the HttpServer:**
    - Instantiate an HttpServer object using HttpServer.create().
    - Provide an InetSocketAddress to specify the listening address and port.
    - Set the backlog to 0 (use the system default).
3. **Define an HttpHandler:**
    - Create a class that implements the HttpHandler interface.

- Override the handle(HttpExchange exchange) method. This is where your request processing logic goes.

4. **Handle Requests within the handle() Method:**
   - Get the request method (e.g., exchange.getRequestMethod()).
   - Get the request URI (e.g., exchange.getRequestURI()).
   - Based on the request method and URI, decide how to respond.
   - Create a response string.
   - Send response headers using exchange.sendResponseHeaders(). Include the status code (e.g., 200 OK) and the response length (in bytes).
   - Get the response body OutputStream using exchange.getResponseBody().
   - Write the response string (converted to bytes) to the OutputStream.
   - Close the OutputStream.
5. **Create a Context:**
   - Use server.createContext() to associate a URL path with your HttpHandler.
6. **Start the Server:**
   - Call server.start() to begin listening for connections.
7. **Handle Different Request Methods (Optional but Recommended):**
   - Inside your handle() method, use if/else statements to handle different request methods (e.g., GET, POST) differently.
   - If a request method is not supported, send back a 405 Method Not Allowed response.
8. **Run and Test:**
   - Run your Java program.
   - Use a web browser or a tool like curl to send requests to your server (e.g., http://localhost:8000/hello).

Example Flow (for a GET request to "/hello"):

1. Client sends a GET request to http://localhost:8000/hello.
2. HttpServer receives the request.
3. HttpServer finds the context associated with "/hello" (which you created).
4. HttpServer calls the handle() method of the associated HttpHandler.
5. Inside handle():
   - The request method is retrieved (it's "GET").
   - The request URI is retrieved (it's "/hello").

    - A response string is created.
    - Response headers (status code 200, content length) are sent.
    - The response body is written to the OutputStream.
    - The OutputStream is closed.
6. HttpServer sends the complete response back to the client.
7. The client receives the response.

This documentation provides a high-level overview of the concepts and process, making it easier to understand the overall structure of a simple HTTP server in Java. It emphasizes best practices and key points to remember.